Ghost Serialization 1.2.0

Complete technical manual — study and reference (A5 / mobile)

Monorepo ghost-serializer · version 1.2.0 · Maven com.ghostserializer · compile-time KSP + reflection-free runtime.

How to read this manual

This document is meant to be studied in PART order (I → VIII) or by jumping via the index (blue links in the PDF). Tables summarize; paragraphs and JSON examples explain the why and when to use each piece. If something does not match your project, open the .kt file indicated under build/generated/ksp/ after compiling.

Navigation index

In the PDF: tap blue entries to jump between sections. In the PDF reader also use the Bookmarks/Outline menu (side panel).

PART I — Fundamentals

1. Ghost philosophy and promise

What problem it solves

When you use Gson, Moshi, or kotlinx.serialization with reflection or runtime metadata, every request pays a cost: finding fields, creating adapters, or walking generic trees. In APIs with thousands of requests per second, that shows up in CPU and allocations.

Ghost inverts the approach: all heavy work happens at compile time, with KSP (Kotlin Symbol Processing). You write a normal data class with one annotation; the compiler generates a UserSerializer.kt file with

concrete code, no reflection.

Compile-time vs runtime (mental diagram)

```
YOUR CODE                    BUILD (KSP)                  RUNTIME
─────────                    ───────────                  ───────
@GhostSerialization    →     UserSerializer.kt      →     Ghost.deserialize<User>(json)
data class User              (fixed code)                 calls UserSerializer directly
```

At runtime Ghost does not do Class.getDeclaredFields(), does not build a dynamic JsonAdapter. It only:

1. Looks up the serializer in cache or registry (a generated when).
2. Passes JSON to a byte-by-byte parser (GhostJsonFlatReader).
3. The generated serializer reads field by field and builds the object.

Quick comparison with other libraries

Aspect	Gson / Moshi (reflectio	kotlinx.serialization	Ghost
:---	:---	:---	:---
When code is generate	Runtime (reflect) or optional KSP	KSP	Always KSP
Dynamic JSON model	Yes	Limited	No (fixed models)
R8 / ProGuard	Broad keep rules	Keep rules	Direct code, little keep
Typical hot path	Adapter map	Encoder/Decoder	Flat reader + monomorphic serializer

The performance promise (in plain language)

• Monomorphic paths: the JIT always sees UserSerializer.deserialize, not a generic interface with 200 implementations. That enables inlining.

• Byte-by-byte parser: JSON lives in a ByteArray; no intermediate DOM tree of objects is created.

• Per-thread pools: reader and writer are reused between calls on the same thread (ThreadLocal on JVM).

• Registry with when: resolving KClass → Serializer is a compiled comparison chain, not a HashMap of strings.

What you should NOT expect from Ghost

• Fully dynamic JSON (Map<String, Any> without a type) like Jackson for admin APIs.

• Changing the schema at runtime without recompiling.

• Replacing Jackson in a Spring project where 90% of DTOs are POJOs without @GhostSerialization.

Ghost shines when your network models are defined and you want maximum performance on Android, JVM server, or KMP iOS.

Complete minimal example

```
// 1. Model (app or shared module)
@GhostSerialization
```

```
data class User(val id: Int, val name: String)

// 2. After compiling, UserSerializer.kt exists (you do not write it)

// 3. Runtime usage
val json = """{"id":1,"name":"Ana"}"""
val user = Ghost.deserialize<User>(json)
val again = Ghost.encodeToString(user)  // {"id":1,"name":"Ana"}
```

If Ghost.deserialize throws NOT_FOUND, it almost always means: missing annotation, KSP did not run, or on iOS missing addRegistry.

PART II — Public API

2. Monorepo module map

Module	Published	Role
ghost-api	**Yes**	**Public annotations, zero deps**
ghost-compiler	**Yes**	**KSP plugin (KotlinPoet)**
ghost-serialization	**Yes KMP**	**Runtime engine**
ghost-gradle-plugin	**Yes**	**Wiring in consumer apps**
ghost-retrofit	**Yes JVM**	**ConverterFactory**
ghost-ktor	**Yes KMP**	**ContentNegotiation ghost()**
ghost-spring-boot-starter	**Yes JVM**	**MVC + WebFlux**
ghost-integration-test	**No**	**Compiler tests**
ghost-benchmark	**No**	**JVM benchmark**
ghost-sample	**No**	**Compose demo**

Typical consumer dependency: ghost plugin + ghost-serialization (via plugin) + ghost-compiler (KSP).

What your project needs (explained)

1. Gradle plugin com.ghostserializer.ghost — configures KSP, adds correct dependencies for Android, JVM, or KMP.

2. ghost-api — annotations only (@GhostSerialization, etc.). Minimal weight.

3. ghost-compiler — KSP processor; generates *Serializer.kt in build/generated/ksp/.

4. ghost-serialization — runtime: Ghost, parsers, writers, primitives.

You do not ship the compiler in your final app as “business logic”; it only runs at compile time. The APK/AAR contains runtime + generated code.

Modules NOT published to Maven Central

• ghost-integration-test: internal regression suite (155 tests in ciTestJvm modules; full Linux ciTest is 642 — see section 23).

- ghost-benchmark: Gson/Moshi/Kotlinx vs Ghost comparisons.
- ghost-sample: Compose demo app.

3. ghost-api — Annotations

Package: com.ghost.serialization.annotations. No external dependencies. Ten public annotations:

Annotation	Target	Role
:---	:---	:---
@GhostSerialization	**class**	**Marks model; discriminator, inferred, name**
@GhostName	**property**	**JSON alias**
@GhostIgnore	**property**	**Omit from JSON**
@GhostEncoder	**property**	**Custom encode function**
@GhostDecoder	**property**	**Custom decode function**
@GhostFlatten	**property**	**Nested JSON → flat property**
@GhostWrap	**property**	**Properties → JSON sub-object**
@GhostResilient	**class/property**	**Tolerance for invalid types**
@GhostFallback	**sealed subclass**	**Default branch**
@GhostSignature	**sealed subclass**	**Signature for inferred polymorphism**

@GhostSerialization (required on models)

Without this annotation on the class, KSP ignores the type. It is the “contract” that you want codegen.

```
@GhostSerialization
data class Product(val id: Long, val title: String)
```

Optional parameters:

Parameter	Default	Meaning
:---	:---	:---
discriminator	**"type"**	**JSON field name indicating subclass in sealed class**
inferred	**false**	**If true, deduces subclass from present fields (no type field)**
name	**class name**	**Registry alias if typeName collides**

Sealed with discriminator (most common case):

```
@GhostSerialization(discriminator = "kind")
sealed class Event {
    @GhostSerialization
    data class Click(val x: Int, val y: Int) : Event()
    @GhostSerialization
    data class Scroll(val delta: Int) : Event()
}
```

Input JSON:

```
{"kind":"Click","x":10,"y":20}
```

The generated serializer reads kind, does when, and calls the Click deserializer.

Sealed inferred (APIs without type field):

```
@GhostSerialization(inferred = true)
sealed class SmartEvent { /* subclasses with unique fields */ }
```

If only Click has x and y, and Scroll only delta, generated code checks which fields appear in JSON and picks the subclass. More generated code, but tolerates legacy APIs.

@GhostName — JSON name ≠ Kotlin name

```
@GhostSerialization
data class ApiUser(
    @GhostName("user_id") val id: Int,
    val name: String
)
```

• In Kotlin you use id.

• In JSON the field is called user_id.

• The generated serializer maps one to the other; you do not need Gson's @SerializedName.

@GhostIgnore — local-only field, never in JSON

```
@GhostSerialization
data class Session(
    val token: String,
    @GhostIgnore val cachedAt: Long = System.currentTimeMillis()
)
```

On serialize, cachedAt does not appear. On deserialize, Kotlin's default is used. Useful for derived or UI fields.

@GhostFlatten — nested JSON, flat property

External API sends:

```
{"settings": {"theme": "dark", "lang": "es"}}
```

Your model wants:

```
@GhostSerialization
data class Config(
    @GhostFlatten("settings.theme") val theme: String,
    @GhostFlatten("settings.lang") val lang: String
)
```

Ghost "flattens" the path when reading/writing. Several properties can point to different paths in the same subtree.

@GhostWrap — several properties → one JSON sub-object

Inverse of flatten. If you have val a: String, val b: Int and want:

```
{"meta": {"a": "x", "b": 1}}
```

```
@GhostWrap("meta") val a: String  // inside meta in JSON
```

@GhostEncoder / @GhostDecoder — types Ghost does not know

For Instant, UUID, types from another library, you define static functions:

```
object DateCoder {
    fun decode(epoch: Long): Instant = Instant.ofEpochSecond(epoch)
    fun encode(instant: Instant): Long = instant.epochSecond
}

@GhostSerialization
data class Log(
    @GhostDecoder(provider = DateCoder::class, function = "decode")
    @GhostEncoder(provider = DateCoder::class, function = "encode")
    val created: Instant
)
```

In JSON created will be a number (epoch). The generated serializer calls DateCoder.decode instead of nextString().

@GhostResilient — do not break on a rare enum

If the backend sends "role": "SUPERADMIN" and your enum only has USER, ADMIN:

• Without resilient: exception.

• With @GhostResilient on the enum or property: null, default value, or skip depending on what KSP generates.

@GhostFallback — sealed with “catch-all”

```
@GhostSerialization
sealed class Payment {
    @GhostSerialization @GhostFallback
    data class Unknown(override val rawType: String) : Payment()
}
```

If the discriminator does not match any known subclass, it deserializes to Unknown instead of failing. Ideal for APIs that add new types without notice.

@GhostSignature — inferred on sealed subclasses

Marks which fields uniquely identify a subclass in inferred polymorphism. The compiler builds a decision tree; without this, inferred on complex sealed types may fail analysis.

PART III — KSP compiler (internals)

4. KSP — Compiler entry point

KSP (Kotlin Symbol Processing) runs during compilation, in rounds. Ghost registers its processor via SPI:

META-INF/services/com.google.devtools.ksp.processing.SymbolProcessorProvider → class GhostSerializationProvider.

What happens when you run ./gradlew build

Step 1 — Discovery: KSP finds all classes annotated with @GhostSerialization in the module.

Step 2 — Analysis (GhostAnalyzer): for each class it validates types, names, flatten, sealed, defaults. If something is invalid, compile error with a message (no silent failure in 1.1.17).

Step 3 — Generation (GhostCodeGenerator + emitters): writes to disk, for example:

build/generated/ksp/<variant>/kotlin/<package>/UserSerializer.kt

Step 4 — Global registry: if there was at least one class, it also generates:

• GhostModuleRegistry_my_app.kt — clazz → serializer map

• META-INF/services/com.ghost.serialization.contract.GhostRegistry — for ServiceLoader on JVM

• META-INF/proguard/ghost-serialization.pro — R8 rules

Step 5 — Normal Kotlin compilation: the Kotlin compiler compiles your sources and generated ones together.

Consumer configuration

```
plugins {
    id("com.ghostserializer.ghost") version "1.2.0"
}

// Optional but recommended with several modules containing models:
ksp {
    arg("ghost.moduleName", "my_app")  // → GhostModuleRegistry_my_app
}
```

If you omit moduleName, the registry is called GhostModuleRegistry_Default. In tests (src/test) a _Test suffix is added to avoid clashing with production.

Where to look at generated code

Platform	Typical path under build/generated/ksp/
:---	:---
Android	build/generated/ksp/androidDebug/kotlin/

JVM	**build/generated/ksp/main/kotlin/**
KMP	**build/generated/ksp/metadata/commonMain/kotlin/**

Open a *Serializer.kt and compare it with PART VIII of this manual (BenchUser walkthrough).

5. GhostAnalyzer and property models

GhostAnalyzer.kt is the compiler “notary”: it turns each KSClassDeclaration into a list of GhostPropertyModel that emitters consume.

Validations it performs (and why they matter)

Validation	If it fails
:---	:---
Only data class, sealed, enum, value class	**Does not generate code for bare interfaces**
Supported types (primitives, String, List, Map nested @GhostSerialization)	**Error: cannot serialize Flow<T> or raw types**
Coherent @GhostFlatten / @GhostWrap paths	**Avoids JSON impossible to read/write**
Unique JSON names per class	**Two fields cannot map to the same key**
Sealed: subclasses also annotated	**Without annotation on child, not in registry**
Custom coder: correct function signature	**decode must be static and types compatible**

What GhostPropertyModel stores (per-field metadata)

Each data class property is summarized in an internal object with:

- kotlinName / jsonName — may differ via @GhostName
- nullable, hasDefault, default value for multi-branch
- bitIndex — position in bit mask (field id → bit 0, name → bit 1, …)
- flags: resilient, flatten path, wrap path, uses external encoder/decoder
- resolved type: enum, sealed, List of X, etc.

That metadata is what DeserializeCodeEmitter uses to write when (selectNameAndConsume) and constructor branches.

Compile-time errors (1.1.17 behavior)

If the analyzer detects a problem, KSP reports an error and the build fails. It does not generate an empty serializer or “best effort”. This protects production: if it compiles, the model was coherent at compile-time.

6. Code generation — Emitters

Class	Role
GhostCodeGenerator	**Orchestrates serialize + deserialize**
SerializeCodeEmitter	**Serialize body (flat + streaming)**
DeserializeCodeEmitter	**Deserialize body**
StandardEmitter	**Normal models**

FragmentedEmitter	**Huge models (>40 fields)**
FragmentedSerializeEmitter	**Avoids JVM method > 64KB**

Constants: GhostEmitterConstants.kt — %L, %T templates, registry names, chunk size 500 for registry when sharding.

Generate serialize pattern (conceptual)

```
// Generated — simplified idea
object UserSerializer : GhostSerializer<User> {
    override val typeName = "User"
    override fun serialize(writer: GhostJsonFlatWriter, value: User) {
        writer.beginObject()
        writer.writeNameHeader(HEADERS[0]) // "id":
        writer.writeInt(value.id)
        writer.writeNameHeader(HEADERS[1]) // "name":
        writer.writeString(value.name)
        writer.endObject()
    }
    override fun deserialize(reader: GhostJsonFlatReader): User {
        reader.beginObject()
        var mask = 0L
        var id = 0
        var name: String? = null
        loop@ while (reader.hasNext()) {
            when (reader.selectNameAndConsume(OPTIONS)) {
                0 -> { id = reader.nextInt(); mask = mask or 1L }
                1 -> { name = reader.nextString(); mask = mask or 2L }
                else -> reader.skipValue()
            }
        }
        reader.endObject()
           if ((mask and REQUIRED) != REQUIRED) throw GhostJsonException("Required field
missing")
        return User(id, name!!)
    }
}
```

selectNameAndConsume — O(1)

JsonReaderOptions precomputes a trie of field names. At runtime there is no == "fieldName" string compare in the main loop hot path.

Bitmask for required fields

Imagine User with required fields id, name, email. The generator assigns:

- id → bit 1 (value 1)
- name → bit 2 (value 2)
- email → bit 4 (value 4)
- REQUIRED_MASK = 7 (1+2+4)

At the end of the loop, if (mask and 7) != 7, some field is missing and GhostJsonException is thrown with message "Required field 'email' missing". A single Long checks all required fields; cheaper than three if (name == null).

FragmentedEmitter (>40 fields)

JVM limits bytecode size per method (~64 KB). A GodObject with 80 fields would produce a huge deserialize. FragmentedEmitter splits the method into several deserializePart1, deserializePart2, etc., transparent to callers.

7. Registry and discovery

Registry sharding (internal)

If a module has more than 500 models (REGISTRY_CHUNK_SIZE), the compiler generates:

• Several methods getShardMap0(), getShardMap1(), ...

• getSerializer(clazz) as a fragmented when to avoid exceeding 64 KB JVM bytecode per method

• lazy allSerializers for prewarm()

Generated file: com.ghost.serialization.generated.GhostModuleRegistry_<moduleName>

GhostRegistry interface:

```
interface GhostRegistry {
    fun <T : Any> getSerializer(clazz: KClass<T>): GhostSerializer<T>?
    fun getAllSerializers(): Map<KClass<*>, GhostSerializer<*>>
}
```

Generated: GhostModuleRegistry_my_app with when (clazz) possibly sharded if there are thousands of models (>500 per method).

JVM/Android discovery:

```
// Ghost.jvm.kt — idea
fun discoverRegistries(): Iterable<GhostRegistry> {
    // Class.forName fast-path + ServiceLoader
}
```

Generated file: META-INF/services/com.ghost.serialization.contract.GhostRegistry

iOS / Kotlin Native: discoverRegistries() returns an empty list (no ServiceLoader). You must register the module at startup:

```
// In the KMP framework init exported to iOS
fun initGhostRuntime() {
    Ghost.addRegistry(GhostModuleRegistry_my_app.INSTANCE)
    Ghost.prewarm()
```

```
}
```

Without this, Ghost.deserialize finds no serializer even though KSP generated the code.

Android/JVM: with ServiceLoader + generated META-INF, you usually do not need addRegistry except for tests or extra manual registries.

addRegistry flow

When you call addRegistry, Ghost walks registry.getAllSerializers() and preloads serializerCache and serializerByName. Call it once at startup, not per request.

---

8. Ghost.kt — Public runtime API

Central file: ghost-serialization/src/commonMain/kotlin/com/ghost/serialization/Ghost.kt

Full public API table

Method	Description
:---	:---
deserialize<T>(json: String)	Decode UTF-8 → flat reader → serializer
deserialize<T>(bytes: ByteArray)	No intermediate String copy
deserialize<T>(source: BufferedSource)	Okio streaming
deserialize(reader) / deserialize(flatReader)	Direct use in generated code
encodeToString<T>(value)	Flat writer → String
encodeToBytes<T>(value)	Flat writer → ByteArray
encodeAndDiscard<T>(value)	Warm-up without result alloc
serialize(sink, value)	Bulk drain to Okio
decodeFromBytes(bytes, KClass)	Spring / Retrofit
decodeFromSource(source, KClass)	Streaming + KClass
encodeToSink(sink, value, KClass)	Encode with dynamic class
getSerializer(KClass) / getSerializer(KType)	Resolution + cache
addRegistry(registry)	Required on iOS
prewarm()	Loads registries + warmUp serializers
getSerializerByName / getSerializerNames	Compiler bridge
throwError(msg)	Uniform IllegalArgumentException
Constants NOT_FOUND, MISSING_ANN	Error messages

Resolution order in getSerializerFromRegistries:

1. mutableRegistries (manual addRegistry)
2. discoverRegistries() — ServiceLoader JVM / empty on iOS
3. registry.getSerializer(clazz) — generated sharded when

Resolving serializer

Order:

1. serializerCache[KClass] (primitives + prewarm)
2. mutableRegistries (manual addRegistry)
3. discoverRegistries() (ServiceLoader JVM)
4. registry.getSerializer(clazz) → generated when

Deserialize — when to use each overload

```
// Most common: HTTP response already as String
val user = Ghost.deserialize<User>(responseBody)

// Better if you already have bytes (Retrofit/OkHttp without intermediate String)
val user = Ghost.deserialize<User>(bytes)

// Large streaming from Okio
val user = Ghost.deserialize<User>(bufferedSource)

// Advanced reader options (coercion, etc.) — requires internal OptIn
Ghost.deserialize<User>(json) { reader ->
    reader.coerceBooleans = true
}
```

Inside: resolveSerializer<User>() → ghostInternalUseFlatReader takes reader from pool → UserSerializer.deserialize(flatReader) → returns object.

Encode — when to use each overload

```
val s = Ghost.encodeToString(order)     // UI, logs, SharedPreferences
val b = Ghost.encodeToBytes(order)      // send over network without String
Ghost.serialize(okioSink, order)        // write directly to socket/file
Ghost.encodeAndDiscard(order)           // only warm JIT, discard bytes
```

serialize(value) without sink is an alias of encodeToString(value) for compatibility.

getSerializer — framework integration

Spring and Retrofit do not always have reified T. They use:

```
Ghost.getSerializer(Order::class)
Ghost.decodeFromBytes(bytes, Order::class)
```

For List<Order> in Retrofit, the factory resolves ParameterizedType and builds ListSerializer(OrderSerializer).

Prewarm — why and when

```
// Application.onCreate (Android) or @PostConstruct (Spring)
Ghost.addRegistry(GhostModuleRegistry_my_app.INSTANCE)      //   only   iOS/KMP   without
ServiceLoader
Ghost.prewarm()
```

prewarm() does three things: discovers registries (JVM), fills serializerCache with all module serializers, and calls warmUp() on each

(dummy deserialize for JIT). The first real request pays less JIT compilation latency.

9. GhostSerializer — Dual contract

Every generated serializer implements GhostSerializer<T> in contract/GhostSerializer.kt.

Why “dual” (two writers and two readers)

Ghost optimizes two distinct scenarios:

1. In-memory / HTTP body as bytes → GhostJsonFlatWriter + GhostJsonFlatReader on contiguous ByteArray. Maximum throughput.

2. Okio streaming (files, chunks) → GhostJsonWriter + GhostJsonReader on BufferedSource.

Generated code implements all four functions. Defaults on the interface may delegate flat ↔ streaming so you do not duplicate logic manually for simple types.

Methods each FooSerializer implements

Method	Main use
:---	:---
serialize(GhostJsonFlatWriter, T)	**encodeToString, Retrofit request**
serialize(GhostJsonWriter, T)	**serialize(sink)**
deserialize(GhostJsonFlatReader)	**Ghost.deserialize(bytes)**
deserialize(GhostJsonReader)	**streaming + options**
warmUp()	**called from prewarm()**
typeName	**readable key ("BenchUser")**
isResilient	**lists that skip bad elements**

Mental example

```
Ghost.encodeToString(user)
    → BenchUserSerializer.serialize(flatWriter, user)

Retrofit response
    → BenchUserSerializer.deserialize(flatReader)
```

Always the same BenchUserSerializer object; different paths depending on buffer type.

PART IV — Internal runtime

End-to-end deserialize and encode pipeline

Deserialize (hot path)

```
JSON bytes
```

```
  → ghostInternalUseFlatReader (ThreadLocal pool)
  → GhostJsonFlatReader.reset(data)
  → FooSerializer.deserialize(reader)
       → beginObject()
       → loop: selectNameAndConsume(OPTIONS)  // O(1) trie
       → nextInt / nextString / skipValue
       → bitmask REQUIRED_MASK
       → primary constructor (multi-branch if applicable)
  → T
```

Encode (hot path)

```
value T
  → Ghost.resolveSerializer<T>()
  → ghostInternalEncodeWithWriter (WriterSinkPair pool)
  → FooSerializer.serialize(flatWriter, T)
       → pre-encoded writeNameHeader
       → writeInt / writeQuotedString
       → endObject()
  → FlatByteArrayWriter.toByteArray()
  → reset() — trims buffer if > maxWarmWriteBufferCapacity
```

Why two writers and two readers

Type	When	Cost
GhostJsonFlatReader	**Ghost.deserialize, Retrofit, Spring body**	**Minimum: contiguous array**
GhostJsonReader	**Okio stream, coerceBooleans options**	**More flexible**
GhostJsonFlatWriter	**encodeToString/Bytes**	**Monomorphic hot path**
GhostJsonWriter	**serialize(sink)**	**Single Okio drain at end**

10. GhostJsonFlatReader — Fast parser

GhostJsonFlatReader is the parser used in almost the entire hot path of Ghost. It reads a ByteArray start to end with a position index, without allocating objects per token.

Lifecycle in a typical deserialize

1. reset(bytes) — points at the response buffer (may be the same array from the pool).

2. beginObject() — expects {, prepares object context.

3. Loop hasNext() / selectNameAndConsume(OPTIONS) — identifies current field.

4. nextInt(), nextString(), etc. — consume value and advance position.

5. skipValue() — if JSON has fields your model does not (forward compatible).

6. endObject() — validates }.

7. Serializer checks mask and builds the data class.

selectNameAndConsume explained without jargon

Instead of comparing strings each iteration (if (name == "email")), the compiler precomputes JsonReaderOptions with a mini-hash of field names. At runtime:

- Returns 0 if the field is "id"
- Returns 1 if it is "name"
- Returns -1 if the object ended
- Returns -2 if unknown field → typically skipValue()

That is amortized O(1) and JIT-friendly (fewer branches with strings).

Security limits

Parameter	What it protects	Typical value
:---	:---	:---
maxDepth	**Infinite nested JSON [[[[**	**255**
maxCollectionSize	**Huge lists/maps (DoS)**	**50k Android, 1M JVM**

If exceeded, clear exception. Does not limit total HTTP body size — configure that in OkHttp/Spring.

Typical generated code

```
reader.beginObject()
while (true) {
    when (reader.selectNameAndConsume(OPTIONS)) {
        0 -> id = reader.nextInt()
        1 -> name = reader.nextString()
        -1 -> break
        -2 -> reader.skipValue()
    }
}
reader.endObject()
```

11. GhostJsonReader — Streaming parser

parser/GhostJsonReader.kt — on Okio / streaming.

- String pool for repeated names
- GhostDiscriminatorPeeker for standard sealed
- Used when API requests GhostJsonReader in options lambda

Pools: ThreadLocal JVM/Android, @ThreadLocal iOS in Ghost.*.kt.

12. Writers — FlatByteArrayWriter

writer/FlatByteArrayWriter.kt + GhostJsonFlatWriter.kt

- Growing flat buffer

• reset() after each encode — releases capacity above maxWarmWriteBufferCapacity

• JVM: 8 MB warm cap; Android/Native: 4 MB; (Wasm sources not published)

```
// After encode, reset keeps buffer up to cap
fun reset() {
    // needsComma=false, depth=0
    // if capacity > maxWarm → shrink to 8KB initial
}
```

Pre-encoded headers: bulk write of "field": without UTF-8 encode per field on hot path.

GhostJsonWriter — Okio variant for serialize(sink).

13. GhostPools and scratch buffers

GhostPools.kt — expect/actual

• acquireScratchBuffer(size) / releaseScratchBuffer

• Size tiers to avoid alloc in number→string conversion and ASCII escape

Instance pools (in Ghost.jvm.kt / .android.kt / .ios.kt):

• GhostJsonFlatReader + ByteArrayGhostSource

• GhostJsonReader

• WriterSinkPair (flat writer + sink)

runSynchronized on registry caches.

14. GhostHeuristics — Platform limits

GhostHeuristics (expect/actual per target) defines compiled limits per platform. They are not flags the user changes in Ghost.deserialize() today.

Row-by-row table

Property	Android	iOS/Native	JVM server	Practical meaning
:---	:---	:---	:---	:---
maxCollectionSize	**50,000**	**500,000**	**1,000,000**	**Max elements when parsing List or Map**
maxWarmWriteBu	**4 MB**	**4 MB**	**8 MB**	**After encode, if internal buffer exceeds this, shrinks to ~8 KB**
maxDepth	**255**	**255**	**255**	**Max nesting of { [**

maxCollectionSize: protects against {"items":[...millions...]}. On JVM server the limit is higher because legitimate batch APIs can be large; on

mobile it is stricter for RAM.

maxWarmWriteBufferCapacity: if you serialize a 6 MB JSON once, the buffer grows; after reset(), Ghost does not keep 6 MB forever on mobile (4 MB cap) but on JVM may keep up to 8 MB to avoid realloc on the next similar response. This improved server write benchmarks without inflating Android RAM.

HTTP body: a 50 MB POST can enter memory whole if OkHttp/Spring allow it; Ghost will parse until maxCollectionSize or system memory says stop. Configure HTTP limits outside Ghost.

Collections supported at runtime (verified in code)

Type	Support
:---	:---
List<T>	Yes — ListSerializer via getSerializer(KType)
Map<String, V>	Yes — MapSerializer
Set<T>	No SetSerializer in ghost-serialization; the README mentions it but the current runtime only resolves List/Map besides primitives and @GhostSerialization models

15. Primitive and collection serializers

serializers/PrimitiveSerializers.kt, ListSerializer, MapSerializer, etc.

Ghost resolves Int, String, List<T>, Map<String,V> with built-in serializers before the registry.

For generics: getSerializer(KType) builds or caches ListSerializer(element).

16. Sealed, inferred, multi-branch

Sealed with discriminator (flow to visualize)

JSON arrives → sealed serializer reads type field (or what you define) → when picks sub-serializer → ClickSerializer.deserialize or ScrollSerializer.deserialize.

Advantage: explicit, easy to debug. Disadvantage: backend must always send the discriminator.

Inferred (no type field)

The compiler analyzes which field combination is unique per subclass. Generates a tree of if/when like: “if x and y present → Click; if only delta → Scroll”. Single JSON pass.

Useful when migrating legacy APIs. More generated bytecode and more branches in the serializer.

Multi-branch constructors (Kotlin defaults)

BenchUser has isActive = true, role = VIEWER, bio = null. They may be missing in JSON. Instead of:

```
// What Ghost does NOT want on hot path
BenchUser(id, name, email, score).copy(isActive = true, role = VIEWER)
```

The compiler generates up to 2^N branches if ((mask and X) == X) return BenchUser(...) (one constructor call per branch) instead of chaining .copy(). Fewer temporary objects, less GC.

PART V — HTTP adapters

17. ghost-gradle-plugin

GhostPlugin.kt when applied:

1. If KSP → adds ghost-compiler to ksp / kspCommonMainMetadata
2. If KMP → ghost-serialization + ghost-api on commonMain
3. If Android/JVM → runtime implementation + api
4. afterEvaluate: if Retrofit/Ktor on classpath → ghost-retrofit / ghost-ktor

```
ghost {
    version.set("1.2.0")
    autoInjectKtor.set(true)
    autoInjectRetrofit.set(true)
}
```

Plugin id: com.ghostserializer.ghost. DEFAULT_VERSION in plugin = 1.2.0.

---

18. ghost-retrofit

Full configuration

```
dependencies {
    implementation("com.ghostserializer:ghost-retrofit:1.2.0")
}

interface ApiService {
    @GET("user/{id}")
    suspend fun getUser(@Path("id") id: Int): User  // User must have @GhostSerialization

    @POST("users")
    suspend fun createUser(@Body user: User): User
}

val retrofit = Retrofit.Builder()
    .baseUrl("https://api.example.com/")
```

```
    .addConverterFactory(GhostConverterFactory.create())  // before Gson if both exist
    .client(okHttpClient)
    .build()
```

What the factory does internally

Response: reads ResponseBody into a reusable buffer (scratch), then ghostInternalUseFlatReader → serializer.deserialize(flatReader). Does not go through intermediate UTF-16 String if the body is already bytes.

Request: ghostInternalEncodeWithWriter { w -> serializer.serialize(w, value) } → RequestBody with application/json.

Cache: ConcurrentHashMap<Type, GhostSerializer> — first call per type resolves; later calls are O(1).

null body: if the Retrofit method allows nullable body and null arrives, converter returns null without serializing.

Coexisting with Gson

If you add GhostConverterFactory before GsonConverterFactory, Retrofit tries Ghost first. Only types with a Ghost serializer use Ghost; the rest fall through to Gson.

---

19. ghost-ktor

GhostContentConverter.kt + GhostKtorExtensions.kt

```
val client = HttpClient {
    install(ContentNegotiation) {
        ghost() // Ktor 2.3.x
    }
}
```

Same pool + flat reader/writer pattern. Ktor 3 in consumer apps may need a custom converter (see android test app GhostKtor3Converter).

---

20. ghost-spring-boot-starter

Dependencies

```
plugins {
    id("com.ghostserializer.ghost") version "1.2.0"
}
dependencies {
    implementation("com.ghostserializer:ghost-spring-boot-starter:1.2.0")
}
```

The ghost plugin in the same module as your @RestController and DTOs with @GhostSerialization.

Auto-configuration (Boot 3.4+)

Three open classes imported via

META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports:

Class	Effect
:---	:---
GhostAutoConfiguration	Marker / imports
GhostWebMvcAutoConfiguration	Registers GhostHttpMessageConverter at start of list
GhostWebFluxAutoConfiguration	Reactive encoder/decoder codecs for WebFlux

No ghost.* in application.properties. Everything is automatic if the DTO has a serializer.

How it interacts with Jackson

Spring Boot brings Jackson by default. Ghost inserts its converter at index 0:

• If Ghost.getSerializer(Order::class) != null → Ghost serializes/deserializes.

• If no Ghost serializer → Jackson continues with the DTO as before.

You can have in the same project: JPA entities with Jackson and API DTOs with Ghost.

Typical controller (no special changes)

```
@RestController
@RequestMapping("/api")
class OrderController(private val service: OrderService) {

    @GetMapping("/orders/{id}")
    fun get(@PathVariable id: Long): OrderDto = service.find(id)

    @PostMapping("/orders")
    fun create(@RequestBody dto: OrderDto): OrderDto = service.create(dto)
}

@GhostSerialization
data class OrderDto(val id: Long, val items: List<LineItem>)
```

Body size limit (important)

Ghost does not cut 100 MB bodies. Configure in Spring:

```
spring:
  codec:
    max-in-memory-size: 10MB
```

And/or limits in nginx/API gateway. See PART VII security model.

Spring Boot 3.4.5 verified in starter tests.

21. ProGuard / R8

Generated: META-INF/proguard/ghost-serialization.pro

Consumer rules in ghost-api. Serializers and generated package must be kept if you use minify — the plugin emits rules.

In Android test app: keep @GhostSerialization and generated.**.

---

22. Maven Central publishing

Publishable (publish.gradle.kts): ghost-* except sample, benchmark, integration-test.

From Mac for full KMP (iosArm64 + iosSimulatorArm64 + JVM + Android AAR).

```
./gradlew publishToSonatype closeAndReleaseSonatypeStagingRepository
```

Coordinates: com.ghostserializer:*:1.2.0

From Linux: iOS variants may be missing on Central.

PART VI — CI, tests, and commands

23. CI and tests

Single workflow: .github/workflows/ci.yml

Job	Runner	Command	What it validates
test-jvm	ubuntu-latest	./gradlew ciTestJvm	Compiler, integration-test, retrofit, ktor, spring, plugin
test-android	ubuntu-latest	:ghost-serialization:te	Android runtime
test-ios	macos-14	kspCommonMainKotlir + iosSimulatorArm64Test	K/N iOS; fails if SKIPPED
publish-check	ubuntu-latest	publishToMavenLocal -PskipTests	Maven packaging

iOS CI: Kotlin/Native does not print "N tests completed". The workflow trusts Gradle exit code and detects SKIPPED/FAILED in the log.

ciTestJvm in root build.gradle.kts:

- ghost-serialization jvmTest
- ghost-compiler test
- ghost-integration-test
- ghost-retrofit, ghost-ktor jvmTest
- ghost-spring-boot-starter test
- ghost-gradle-plugin test

GitHub jobs: JVM, Android testDebugUnitTest, iOS macos-14.

Verified on this repo (./gradlew ciTestJvm, May 2026): 416 JVM-module tests. Full ./gradlew ciTest on Linux: 642 (416 ciTestJvm + 226 testDebugUnitTest Android). On macOS with Xcode: ~874 (642 + ~232 iosSimulatorArm64Test per README).

ghost-benchmark:run depends on ciTest (except -PskipTests).

Reference Gradle commands

Goal	Command
:---	:---
Local CI (JVM)	./gradlew ciTestJvm
Full CI (+ Android)	./gradlew ciTest
iOS (Mac only)	./gradlew :ghost-serialization:iosSimulatorArm64Test
View generated KSP code	./gradlew :ghost-integration-test:compileKotlin
Benchmark	./gradlew :ghost-benchmark:run --args="--runs 10000 --warmup 20000"
Benchmark without tests	./gradlew :ghost-benchmark:run -PskipTests --args="--runs 10000 --warmup 20000"
Sample Android	./gradlew :ghost-sample:assembleDebug
Publish local	./gradlew publishToMavenLocal -PskipTests
Maven Central	./gradlew publishToSonatype closeAndReleaseSonatypeStagingRepository
Regenerate manual PDF	.venv-pdf/bin/python scripts/build_ghost_manual_pdf.py

Toolchain: JDK 17, Kotlin/KSP per gradle/libs.versions.toml.

24. Test apps (your 3 repos)

App	What it validates
ghost-android-test-app	Gson/Moshi/KSer vs Ghost, Retrofit, Ktor3
ghost-spring-boot-test-app	Jackson vs Ghost WebFlux, benchmark.py
ghost-ios-test-app	XCFramework + GhostBridge + Codable

All use 1.2.0 Maven Central (no mavenLocal in final config).

25. End-to-end flow on Android

Step by step (from scratch)

1. settings.gradle.kts — pluginManagement { gradlePluginPortal() }
2. app/build.gradle.kts — id("com.ghostserializer.ghost") version "1.2.0"
3. Create data class with @GhostSerialization in the network package
4. Build → Make Project — verify UserSerializer.kt exists in app/build/generated/ksp/
5. Application.onCreate: Ghost.prewarm() (optional but recommended for

high-traffic apps)

6. Retrofit: .addConverterFactory(GhostConverterFactory.create())

7. Release: confirm ProGuard keep includes com.ghost.serialization.generated.**

What happens on the first Retrofit request

```
OkHttp receives bytes
  → GhostConverterFactory
  → ghostInternalUseFlatReader
  → UserSerializer.deserialize
  → User object in memory
```

No reflection on that path.

---

26. End-to-end flow on iOS

1. KMP shared module with ghost plugin + export serialization/api
2. assembleXCFramework on Mac
3. Swift imports framework
4. Ghost.addRegistry(INSTANCE) in Kotlin bridge
5. deserialize via bridge — prefer UTF-8 String vs raw bytes

---

27. End-to-end Spring flow

1. implementation ghost-spring-boot-starter
2. ghost plugin + ksp in same module as controllers
3. Annotated DTOs
4. Normal controllers @RequestBody / @ResponseBody
5. Ghost converter only for types with serializer

PART VII — Publishing, security, errors

Security model and limits

Ghost separates three policy layers:

1. Parser (Ghost): maxCollectionSize, maxDepth — anti-DoS on lists/maps and nesting.

2. Encode memory: maxWarmWriteBufferCapacity — does not retain huge buffers between requests.

3. HTTP (your app): body size, timeouts, reverse proxy.

A 100 MB HTTP body is not rejected automatically by Ghost; configure OkHttp, spring.codec.max-in-memory-size, nginx.

GhostJsonException includes approximate position in the buffer to debug malformed JSON or missing required fields.

28. Common errors

NOT_FOUND or “No serializer found”

Typical message: includes NOT_FOUND and the type’s KClass.

Causes and fix:

1. Class lacks @GhostSerialization → add annotation and recompile.

2. KSP did not run (broken clean, module without ghost plugin) → ./gradlew :app:kspDebugKotlin or rebuild.

3. Model in another module without KSP in that module → each module with models needs plugin + ksp.

4. iOS/KMP: forgot Ghost.addRegistry(GhostModuleRegistry_xxx.INSTANCE) at framework startup.

5. R8 removed serializers → add generated.** and @GhostSerialization rules.

GhostJsonException — missing required field

Message: Required field 'email' missing in JSON

Means: JSON did not include a field that has no Kotlin default. Not a Ghost bug: API contract mismatch.

What to do: fix backend, make field nullable, or give Kotlin default.

GhostJsonException — malformed JSON

Position in the message points to approximate byte of error (missing comma, unclosed string). Useful to log raw request body.

maxCollectionSize exceeded

List or map in JSON exceeded platform limit (50k on Android). May be legitimate large JSON or DoS attack.

What to do: paginate API, or if you control the reader in custom decoder, adjust maxCollectionSize on that path (not on global pooled Ghost.deserialize).

Spring still uses Jackson for my DTO

GhostHttpMessageConverter only handles classes where Ghost.getSerializer(clazz) != null. If your @RequestBody Data lacks @GhostSerialization in the same module compiled with KSP, Jackson wins.

Fix: annotate DTO, apply ghost plugin to controller module, verify Ghost

converter is registered (starter does index 0).

Plugin com.ghostserializer.ghost not found

Gradle does not resolve the plugin. Check pluginManagement in settings.gradle.kts with gradlePluginPortal(), version 1.2.0 on Maven Central, and sync again.

iOS: works in debug, fails in release

Almost always R8/ProGuard or missing addRegistry. Check generated rules in META-INF/proguard/ of the module with KSP.

29. Quick API — cheat sheet

```
Ghost.deserialize<T>(json: String)
Ghost.deserialize<T>(bytes: ByteArray)
Ghost.encodeToString(value)
Ghost.encodeToBytes(value)
Ghost.serialize(sink, value) // Okio bulk write
Ghost.addRegistry(registry)  // required on iOS
Ghost.prewarm()
Ghost.getSerializer(MyClass::class)
```

30. Key files to read in the repo

Topic	Path
:---	:---
API	**ghost-api/src/commonMain/kotlin/com/ghost/serializat**
Processor	**ghost-compiler/src/main/kotlin/com/ghost/serialization**
Generator	**ghost-compiler/src/main/kotlin/com/ghost/serialization**
Runtime	**ghost-serialization/src/commonMain/kotlin/com/ghost/**
Flat reader	**ghost-serialization/src/commonMain/kotlin/com/ghost/**
Flat writer	**ghost-serialization/src/commonMain/kotlin/com/ghost/**
JVM pools	**ghost-serialization/src/jvmMain/kotlin/com/ghost/seria**
iOS	**ghost-serialization/src/iosMain/kotlin/com/ghost/serial**
Retrofit	**ghost-retrofit/src/main/kotlin/com/ghost/serialization/**
Ktor	**ghost-ktor/src/commonMain/kotlin/com/ghost/serializa**
Spring	**ghost-spring-boot-starter/src/main/kotlin/com/ghost/s**
Plugin	**ghost-gradle-plugin/src/main/kotlin/com/ghost/gradle/**

31. Version 1.1.17 — relevant changes

- No maxPayloadBytes in core (HTTP limits in OkHttp/Spring/nginx)
- JVM warm buffer 8 MB (fix inflated WRITE benchmark)
- Spring split open AutoConfiguration (Boot 3.4)
- Centralized ciTestJvm
- SpringBootTest in starter

• CI: 416 (ciTestJvm) / 642 (ciTest on Linux) — verified by Gradle test XML

32. Glossary

• KSP: Kotlin Symbol Processing — compile-time codegen
• Flat path: reader/writer on contiguous ByteArray
• Registry: clazz → generated serializer map
• Shard: split registry when for JVM limit
• Trie options: O(1) structure for JSON field names
• ServiceLoader: JVM discovery of GhostRegistry

33. GhostSerializationProcessor — real code

File: ghost-compiler/src/main/kotlin/com/ghost/serialization/compiler/GhostSerializationProcessor.kt

The processor is the heart of compile-time. On each KSP round:

```
override fun process(resolver: Resolver): List<KSAnnotated> {
    val symbols = resolver.getSymbolsWithAnnotation(
        C.STR_ANNOTATION_SERIALIZATION
    )
    val validClasses = symbols.filterIsInstance<KSClassDeclaration>().toList()
    validClasses.forEach { processClass(it) }
    if (validClasses.isNotEmpty()) {
        generateModuleRegistry()
        generateProGuardRules()
        generateServiceFile()
    }
    return unableToProcess.toList()
}
```

Registry name: KSP option ghost.moduleName → GhostModuleRegistry_<name>. If code lives in src/test, _Test suffix is added to avoid colliding with production registry.

Per annotated class outputs:

• com.your.package.UserSerializer (object implementing GhostSerializer<User>)
• Entry in internal registry map

Global outputs (once per module):

• GhostModuleRegistry_my_app.kt with INSTANCE
• META-INF/services/com.ghost.serialization.contract.GhostRegistry
• META-INF/proguard/ghost-serialization.pro

34. JsonReaderOptions — O(1) trie in detail

parser/JsonReaderOptions.kt precomputes a 1024-slot dispatch table.

```
class JsonReaderOptions(
    internal val rawBytes: Array<ByteArray>,
    internal val shift: Int,
    internal val multiplier: Int
) {
    internal val dispatch = IntArray(1024) { -1 }
    // init: hash of up to 4 bytes of field name → index
}

// In generated serializer:
private val OPTIONS = JsonReaderOptions.of(
    0, 31, "id", "name", "email"
)
```

In the deserialize loop:

```
when (reader.selectNameAndConsume(OPTIONS)) {
    0 -> id = reader.nextInt()
    1 -> name = reader.nextString()
    -1 -> break      // end of object
    -2 -> reader.skipValue()  // unknown field
}
```

Why it matters: comparing strings on every field of a large JSON destroys performance. Ghost compares buffer bytes with names precomputed in rawBytes.

35. Ghost.jvm.kt — pools and discovery

ThreadLocal pools (same idea on Android):

```
private val flatReaderPool = ThreadLocal<GhostJsonFlatReader>()
private val writerPool = ThreadLocal<WriterSinkPair>()

actual fun ghostInternalEncodeWithWriter(
    block: (GhostJsonFlatWriter) -> Unit
): ByteArray {
    val pair = acquireFlatWriterPair()
    block(pair.writer)
    val result = pair.byteWriter.toByteArray()
    pair.byteWriter.reset()
    return result
}
```

discoverRegistries() on JVM:

1.

Class.forName("com.ghost.serialization.generated.GhostModuleRegistry_Default").getField("INSTANCE") (JVM fast path)

2. ServiceLoader.load(GhostRegistry::class.java) reads META-INF/services

That is why on Android/JVM you do not need addRegistry if KSP generated the service file and R8 did not remove it.

36. Ghost.ios.kt — critical differences

```
actual fun discoverRegistries(): Iterable<GhostRegistry> = emptyList()
```

iOS/Native does not use ServiceLoader. You must register manually at KMP framework startup:

```
// shared/src/iosMain or bridge
fun initGhost() {
    Ghost.addRegistry(GhostModuleRegistry_my_app.INSTANCE)
    Ghost.prewarm()
}
```

Pools with @ThreadLocal instead of java.lang.ThreadLocal.

Synchronization with objc_sync_enter/exit for shared caches.

Framework export: the module with models + KSP must export ghost-serialization and ghost-api in XCFramework export(...).

37. GhostConverterFactory — Retrofit flow

```
class GhostConverterFactory private constructor() : Converter.Factory() {

                override    fun    responseBodyConverter(type,    annotations,    retrofit):
Converter<ResponseBody, *>? {
        val serializer = getSerializerWithCache(type) ?: return null
        return Converter { body ->
            body.use {
                // Read stream → scratch buffer → flat reader
                ghostInternalUseFlatReader(scratch, offset) { reader ->
                    serializer.deserialize(reader)
                }
            }
        }
    }

         override  fun  requestBodyConverter(type,  annotations,  retrofit):  Converter<*,
RequestBody>? {
        val serializer = getSerializerWithCache(type) ?: return null
        return Converter { value ->
            ghostInternalEncodeWithWriter { w ->
                serializer.serialize(w, value)
```

```
            }.toRequestBody(MEDIA_TYPE)
        }
    }
}
```

• getSerializerWithCache supports List<T> and Map<String,V> via ParameterizedType
• If no serializer → null → Retrofit tries another converter
• Media type: application/json

38. Ktor integration — ghost()

GhostContentConverter.kt implements Ktor's ContentConverter.

```
fun ContentNegotiation.Config.ghost() {
    register(ContentType.Application.Json, GhostContentConverter())
}
```

Same flat reader/writer path as Retrofit. The Gradle plugin can inject ghost-ktor automatically if it detects Ktor on classpath (autoInjectKtor).

Ktor 3: some apps use a different API; the android test app includes GhostKtor3Converter as reference if ghost() does not compile.

39. Spring — three auto-configurations

Class	Activates
GhostAutoConfiguration	**Marker / imports**
GhostWebMvcAutoConfiguration	**GhostHttpMessageConverter in MVC**
GhostWebFluxAutoConfiguration	**Reactive codecs**

GhostHttpMessageConverter:

```
override fun supports(clazz: Class<*>) =
    Ghost.getSerializer(clazz.kotlin) != null

override fun readInternal(clazz: Class<*>, inputMessage: HttpInputMessage) =
    Ghost.decodeFromBytes(inputMessage.body.readBytes(), clazz.kotlin)

override fun writeInternal(t: Any, outputMessage: HttpOutputMessage) {
    val ser = Ghost.getSerializer(t::class as KClass<Any>)!!
    val bytes = ghostInternalEncodeWithWriter { w ->
        ser.serialize(w, t)
    }
    outputMessage.body.write(bytes)
}
```

The converter is inserted at index 0 to take priority over Jackson on Ghost types.

40. integration-test sample models

Package:
ghost-integration-test/src/main/kotlin/com/ghost/serialization/integration/model/

Types to study in the repo (sources, not generated):

- BenchUser — typical user benchmark
- ComplexObject — deep nesting
- SmartEvent — sealed inferred
- DeviceCommand — sealed with discriminator
- HugeModel — >40 fields → FragmentedEmitter
- DeepNestedModel — JSON depth

After ./gradlew :ghost-integration-test:compileKotlin, open:

ghost-integration-test/build/generated/ksp/main/kotlin/

There you will see real *Serializer.kt and GhostModuleRegistry_Default.kt.

41. FragmentedEmitter — huge models

When a data class exceeds ~40 properties, the compiler splits serialize/deserialize into fragments to avoid the 64 KB JVM bytecode limit per method.

Symptom if it did not exist: compile error on generated serializer for massive models (legacy APIs, ERP DTOs).

42. Multi-branch constructors (≤4 properties with default (MAX_DEFAULT_BRANCH_COUNT = 4))

If a constructor has up to 3 parameters with default, KSP can generate when (mask) branches that call the primary constructor once with the correct argument combination, avoiding:

```
// Anti-pattern Ghost avoids on hot path
User(id).copy(name = n)
```

This reduces allocations on deserialize of objects with many optional fields.

43. Sealed inferred vs discriminator

With discriminator (discriminator = "type"):

```
{"type":"click","x":10,"y":20}
```

Inferred (inferred = true): compiler generates decision tree from which

fields appear:

```
{"x":10,"y":20}
```

No type field. Useful for inconsistent APIs; more generated code and more serializer branches.

44. Resilient and Fallback at runtime

@GhostResilient on enum: unknown JSON value → null or first value per generated policy.

@GhostFallback on sealed: default subclass if discriminator does not match.

GhostSerializer.isResilient: lists that tolerate malformed elements (skip instead of abort).

45. ghostInternalUseFlatReader — decode pipeline

```
inline fun <T> ghostInternalUseFlatReader(
    bytes: ByteArray,
    limit: Int = bytes.size,
    block: (GhostJsonFlatReader) -> T
): T {
    // Acquire reader from pool, reset(bytes), run block, return
}
```

Ghost.deserialize<T>(json) converts String → UTF-8 bytes → flat reader → serializer.deserialize(reader).

Ghost.deserialize<T>(bytes) skips String conversion if you already have bytes (Retrofit, OkHttp).

46. encodeToString vs serialize(sink)

API	Output	Use
encodeToString	**UTF-8 String**	**UI, logs, tests**
encodeToBytes	**ByteArray**	**network, binary cache**
serialize(sink)	**Okio BufferedSink**	**streaming to socket/file without full intermediate copy**

All use GhostJsonFlatWriter + FlatByteArrayWriter internally on hot path.

47. Gradle — dependencies the plugin injects

KMP commonMain:

- com.ghostserializer:ghost-serialization
- com.ghostserializer:ghost-api (api)

KSP:

• com.ghostserializer:ghost-compiler on ksp / kspCommonMainMetadata / targets

Conditional afterEvaluate:

• Retrofit on classpath → ghost-retrofit

• Ktor on classpath → ghost-ktor

```
ghost {
    version.set("1.2.0") // or omit if plugin brings DEFAULT_VERSION
}
```

48. R8 / ProGuard — generated rules

The processor writes rules that keep:

• Generated *Serializer classes

• GhostModuleRegistry_* and its INSTANCE field

• GhostRegistry interfaces for ServiceLoader

On Android consumer:

```
-keep @com.ghost.serialization.annotations.GhostSerialization class * { *; }
-keep class com.ghost.serialization.generated.** { *; }
```

Without this: NOT_FOUND in release even if debug works.

49. Benchmark and CI — useful commands

```
# Monorepo JVM tests (like CI)
./gradlew ciTestJvm

# Benchmark (requires prior tests except -PskipTests)
./gradlew :ghost-benchmark:run

# Regenerate serializers locally
./gradlew :ghost-integration-test:compileKotlin

# PDF of this manual
.venv-pdf/bin/python scripts/build_ghost_manual_pdf.py
```

PDF output: docs/Ghost-Serialization-Manual-1.2.0.pdf (A5 format).

50. Study checklist (40 min on phone)

1. Read ch. 1–4 (philosophy + annotations)
2. Read ch. 6–7 (generated serializer + registry)
3. Read ch. 9–12 (Ghost API + flat reader/writer)
4. Read ch. 35–36 (JVM vs iOS — critical)
5. Adapter for your stack: 37 Retrofit, 38 Ktor, 39 Spring

6. On PC open a generated *Serializer.kt and compare with ch. 6
7. Review ch. 28 common errors before production integration

PART VIII — integration-test catalog and generated code

Module :ghost-integration-test is the compiler and runtime regression lab. After ./gradlew :ghost-integration-test:compileKotlin, inspect:

ghost-integration-test/build/generated/ksp/main/kotlin/

VIII.1 Full model catalog

96 types annotated with @GhostSerialization in src/main + src/test (includes sealed subclasses). 26 test classes in src/test/kotlin/com/ghost/serialization/integration/.

A. Benchmark and real API shape

Model	File	What it validates
:---	:---	:---
BenchUser	**BenchUser.kt**	**Multi-branch (3 defaults), nested enum**
BenchResult	**BenchResult.kt**	**Benchmark response**
BenchmarkMetrics	**BenchmarkMetrics.kt**	**Aggregated metrics**
ComplexResponse	**ComplexResponse.kt**	**Composite API payload**
ApiUserEvent	**ApiUserEvent.kt**	**User event**
ApiProductConfig	**ApiProductConfig.kt**	**Product config**
ExtremeMetadata	**ExtremeMetadata.kt**	**Bulky metadata**
Address	**Address.kt**	**Simple nesting**
Category	**Category.kt**	**Catalog**
Tag	**Tag.kt**	**Tags**
Priority	**Priority.kt**	**Priority enum**
UserRole	**UserRole.kt**	**Role enum (used by BenchUser)**

Test: GhostStressTest, GhostLibraryMethodTest

B. Nesting, graphs, and generics

Model	What it validates
:---	:---
ComplexObject	**Deep multi-level object**
NestedContainer	**Nested container**
RecursiveNode	**Controlled recursion**
RecursiveGraphNode	**Recursive graph**
NestedGenericModel	**Nested generics**
DeepGenericModel	**Deep generics**
MapEdgeCaseModel	**Map key/value edge cases**

Test: GhostRobustnessTest, GhostAdvancedTypesTest, GhostTypeSystemTest

C. JVM fragmentation (>40 fields)

Model	Approx. fields	Emitter
:---	:---	:---
Object40	40	StandardEmitter (limit)
Object41	41	FragmentedEmitter
GodObject	massive	FragmentedEmitter + FragmentedSerializeEmitter
WideModel	wide	Serialize fragmentation

Test: GhostBoundaryFragmentationTest, GhostGodObjectTest

D. Structural transformations

Model	Key annotation
:---	:---
FlattenedModel	@GhostFlatten
DeepFlattenedModel	deep flatten
WrappedModel	@GhostWrap
MixedStructuralModel	mix flatten/wrap
WrapSharedPathModel	shared paths
StructuralCollisionModel	path collisions
CollisionChild	collision child
CollisionModel	key collision

Test: GhostStructuralTransformationTest

E. Polymorphism with discriminator

Root / subclasses	Discriminator
:---	:---
SmartDevice → Light, Thermostat, UnknownD	type + @GhostFallback
ApiEventDefault → Login, Logout	implicit default
ApiEventExplicitType → Login, Logout	explicit type
GhostKindEvent → Created, Deleted, Updated	kind
StripeObject → Charge, Refund	object
JsonLdNode → Person, Organization	JSON-LD @type
GhostShape → Circle, Square	geometric sealed
GhostDiscriminatorTestPayload	test payload

Test: GhostCustomDiscriminatorTest, GhostResilienceAndFallbackTest

F. Inferred polymorphism (test sources only)

Model	Test
:---	:---
SmartEvent, TempEvent, HumidityEvent, MixedEvent, MotionEvent	GhostInferredPolymorphismTest
DeviceCommand → Reboot, SetBrightness, UpdateFirmware	GhostInferredPolymorphismTest

InferredNestedContainer	**nested inferred**
HugeModel, DeepNestedModel, MassiveInferredRoot	**GhostProductionHardeningTest**

Generated under: build/generated/ksp/test/kotlin/

G. Resilience, coercion, and enums

Model	Behavior
:---	:---
ResilientItem	**@GhostResilient items**
DeepResilientModel	**deep resilient**
SmartHome, HomeConfig, HomeStatus	**IoT resilient**
ResilientEnumModel	**unknown enum**
CoercionStressModel	**strings→numbers**
BooleanCoercionModel	**0/1 → bool**
GhostStandardsEnum, GhostEnumWrapper	**enum standards**

Test: GhostCoercionTest, GhostEnumResilienceTest, GhostEnumStandardsTest

H. Custom coders and external types

Model	Mechanism
:---	:---
CustomCoderStressModel	**@GhostEncoder/@GhostDecoder stress**
CustomDateUser, LegacyUser	**legacy dates**
EncoderHexUtils, EncoderDateUtils, EncoderBooleanUtils	**providers**
ContextualModel, ModelWithExternal	**3rd-party types**
ExternalColor + ExternalColorSerializer	**manual serializer**
ExternalDate + ExternalDateSerializer	**contextual date**

Test: GhostCustomCoderTest, GhostContextualTest

I. Value classes and tokens

Model	Detail
:---	:---
UserId	**@JvmInline value class**
UserWithValueClass	**transparent wrapper**
GhostUserToken	**typed token**

Test: GhostValueClassTest

J. Nullability, defaults, evolution

Model	Detail
:---	:---
NullabilityStressModel	**null on all types**
DefaultValueNullModel	**default + null**
NullablePrimitives	**nullable primitives**
CollectionOfNulls	**List with nulls**
EvolutionModel	**new/old fields**

IgnoreModel	**@GhostIgnore**

Test: GhostNullabilityStressTest, GhostHardeningTest, GhostFinalHardeningTest

K. Naming, Unicode, reserved words

Model	Detail
:---	:---
NamingModel	**@GhostName aliases**
ReservedWordModel	**reserved JSON names**
UniCodeModel	**complex UTF-8**
EmojiKeyModel	**emoji keys**

Test: GhostFeatureTest, GhostEliteHardeningTest

L. Malice, DoS, and large strings

Model	Detail
:---	:---
MaliceModel	**malicious payloads**
LargeStringModel, LargeStringData	**huge strings**
StressMetrics	**stress metrics**
DecimalStress	**decimal precision**

Test: GhostMaliceTest, GhostProductionHardeningTest

M. Concurrency and multi-branch

Topic	Detail
:---	:---
BenchUser	**3 parameters with default → 8 constructor branches**
GhostConcurrencyExpansionTest	**pools + threads**

Test: GhostMultiBranchConstructorTest, GhostConcurrencyExpansionTest

Test class → responsibility map

Test class	Focus
:---	:---
GhostStressTest	**throughput / stability**
GhostRobustnessTest	**rare types**
GhostBoundaryFragmentationTest	**40/41 fields**
GhostGodObjectTest	**GodObject**
GhostStructuralTransformationTest	**flatten/wrap**
GhostInferredPolymorphismTest	**inferred**
GhostCustomDiscriminatorTest	**custom discriminator**
GhostResilienceAndFallbackTest	**resilient + fallback**
GhostCoercionTest	**type coercion**
GhostCustomCoderTest	**encoder/decoder**
GhostContextualTest	**external**
GhostValueClassTest	**inline classes**
GhostNullabilityStressTest	**nulls**

GhostMaliceTest	**parser security**
GhostProductionHardeningTest	**massive inferred**
GhostMultiBranchConstructorTest	**bitmask branches**
GhostConcurrencyExpansionTest	**threads**
GhostLibraryMethodTest	**API parity**
GhostTypeSystemTest	**Map/List edge**
GhostAdvancedTypesTest	**generics**
GhostFeatureTest	**naming**
GhostEliteHardeningTest	**unicode**
GhostHardeningTest / GhostFinalHardeningTe	**evolution**
GhostEnumResilienceTest / GhostEnumStandardsTest	**enums**

VIII.2 Walkthrough: BenchUserSerializer line by line

Source model (KSP input)

```
@GhostSerialization
data class BenchUser(
    val id: Int,
    val name: String,
    val email: String,
    val score: Double,
    val isActive: Boolean = true,
    val role: UserRole = UserRole.VIEWER,
    val bio: String? = null
)
```

Required JSON fields: id, name, email, score (bits 1+2+4+8 = mask 15L).

Optional with Kotlin default: isActive, role, bio → compiler generates multi-branch (8 constructor combinations).

Generated output (location)

build/generated/ksp/main/kotlin/com/ghost/serialization/integration/model/BenchUserSerializer.kt

Block 1 — Header and contract (L1–38)

```
@file:Suppress("UNUSED_VARIABLE", "UNCHECKED_CAST", "UNUSED_EXPRESSION", ...)
@file:OptIn(InternalGhostApi::class)

public object BenchUserSerializer : GhostSerializer<BenchUser> {
  override val typeName: String = "BenchUser"
```

- object (not class): zero alloc on lookup; singleton.
- typeName: key in GhostModuleRegistry and getSerializerByName.

Block 2 — JsonReaderOptions (L40–48)

```
private val OPTIONS = JsonReaderOptions.of(0, 31,
```

```
      "id", "name", "email", "score", "isActive", "role", "bio")
```

• 1024-slot trie; hash of first 4 bytes of field name.

• At runtime: selectNameAndConsume(OPTIONS) returns index 0..6, -1 end, -2 unknown.

Block 3 — Pre-encoded headers (L50–62)

```
private val H_ID: ByteString = "\"id\":".encodeUtf8()
private val H_EMAIL: ByteString = "\"email\":".encodeUtf8()
private val H_NAME: ByteString = "\"name\":".encodeUtf8()
// (and H_SCORE, H_ISACTIVE, H_ROLE, H_BIO the same way)
```

• Serialize writes "id": without re-encoding UTF-8 per request.

• writeField(H_ID, value.id) concatenates header + value.

Block 4 — Deserialize: variables and mask (L67–76)

```
var _id: Int = 0
var _name: String? = null
var _role: UserRole? = null
var _bio: String? = null
var _mask0 = 0L
reader.beginObject()
```

• One temporary variable per property.

• _mask0 accumulates bits: id=1, name=2, email=4, score=8, isActive=16, role=32, bio=64.

Block 5 — Field loop (L77–114)

```
while (true) {
  val index = reader.selectNameAndConsume(OPTIONS)
  when (index) {
    0 -> { _id = reader.nextInt(); _mask0 = _mask0 or 1L }
    1 -> { _name = reader.nextString(); _mask0 = _mask0 or 2L }
    5 -> { _role = UserRoleSerializer.deserialize(reader); _mask0 = _mask0 or 32L }
    6 -> { _bio = if (reader.isNextNullValue()) { reader.consumeNull(); null }
               else reader.nextString(); _mask0 = _mask0 or 64L }
    -1 -> break
    -2 -> reader.skipValue()
  }
}
reader.endObject()
```

• Delegation: UserRole uses its own UserRoleSerializer (generated enum).

• Nullable: explicit isNextNullValue branch for bio.

• Unknown: skipValue() forward-compatible.

Block 6 — Required field validation (L116–126)

```
if ((_mask0 and 15L) != 15L) {
  if ((_mask0 and 1L) == 0L) reader.throwError("Required field 'id' missing in JSON")
```

```
   else if ((_mask0 and 2L) == 0L) reader.throwError("Required field 'name' missing in JSON")
  // (same for email and score)
}
```

• One AND on aggregated mask; specific message for first missing field.

• Throws GhostJsonException with buffer position.

Block 7 — Multi-branch constructors (L127–200)

```
if ((_mask0 and 112L) == 112L) {
  return BenchUser(id=_id, name=_name!!, email=_email!!, score=_score,
      isActive=_isActive, role=_role!!, bio=_bio)
}
if ((_mask0 and 48L) == 48L) {
  return BenchUser(id=_id, name=_name!!, email=_email!!, score=_score, isActive=_isActive, role=_role!!)
}
// Other branches: 112L (all optional), 80L, 96L, 16L, 32L, 64L...
return BenchUser(id=_id, name=_name!!, email=_email!!, score=_score)  // required only
```

• 112L = 16+32+64 → isActive + role + bio present.

• Each branch calls the primary constructor once with the exact combination of arguments present in JSON.

• Avoids chained .copy() on hot path (1.1.16+ optimization).

Block 8 — Flat deserialize (L206–340)

Duplicate logic for GhostJsonFlatReader — same structure, different reader (contiguous byte array). Ghost.deserialize(bytes) uses this path via pool.

Block 9 — Streaming and flat serialize (L342–370)

```
override fun serialize(writer: GhostJsonFlatWriter, value: BenchUser) {
  writer.beginObject()
  writer.writeField(H_ID, value.id)
  writer.writeField(H_NAME, value.name)
  writer.writeField(H_EMAIL, value.email)
  writer.writeField(H_SCORE, value.score)
  writer.writeField(H_ISACTIVE, value.isActive)
  writer.name(H_ROLE)
  UserRoleSerializer.serialize(writer, value.role)
  if (value.bio != null) {
    writer.writeField(H_BIO, value.bio)
  }
  writer.endObject()
}
```

• Nullable optional fields: omitted if null (bio).

• Fields with Kotlin default but present: always serialized if not null.

Block 10 — warmUp() (L372–385)

```
override fun warmUp() {

deserialize(GhostJsonReader("""{"id":0,"name":"","email":"","score":0.0}""".encodeToByteArr
ay()))

deserialize(GhostJsonFlatReader("""{"id":0,"name":"","email":"","score":0.0}""".encodeToByt
eArray()))
}
```

• Ghost.prewarm() invokes this to JIT/compile paths before real traffic.

Full flow in your app

```
Ghost.deserialize<BenchUser>(json)
  → resolveSerializer → BenchUserSerializer (registry)
  → ghostInternalUseFlatReader(bytes) { reader ->
       BenchUserSerializer.deserialize(reader)  // blocks 5–7
     }
```

VIII.3 integration-test generated registry

File: build/generated/ksp/main/kotlin/com/ghost/serialization/generated/GhostModuleRegistry_Default.kt

```
// Conceptual structure (simplified)
class GhostModuleRegistry_Default : GhostRegistry {
  companion object { val INSTANCE = GhostModuleRegistry_Default() }

  override fun <T : Any> getSerializer(clazz: KClass<T>): GhostSerializer<T>? =
    when (clazz) {
      BenchUser::class -> BenchUserSerializer as GhostSerializer<T>
      Address::class -> AddressSerializer as GhostSerializer<T>
      BenchUser::class -> BenchUserSerializer as GhostSerializer<T>
      Address::class -> AddressSerializer as GhostSerializer<T>
      else -> null
    }

  override fun getAllSerializers(): Map<KClass<*>, GhostSerializer<*>> by lazy {
    // full map for prewarm()
  }
}
```

ServiceLoader: META-INF/services/com.ghost.serialization.contract.GhostRegistry points to this class.

Test registry: models in src/test generate GhostModuleRegistry_Default_Test (_Test suffix when

ghost.moduleName is Default and file is in test/).

Local inspection:

```
./gradlew :ghost-integration-test:compileKotlin
ls ghost-integration-test/build/generated/ksp/main/kotlin/com/ghost/serialization/
```

51. Maven artifacts table 1.2.0

```
com.ghostserializer:ghost-api
com.ghostserializer:ghost-serialization
com.ghostserializer:ghost-compiler          (KSP)
com.ghostserializer:ghost-gradle-plugin
com.ghostserializer:ghost-retrofit
com.ghostserializer:ghost-ktor
com.ghostserializer:ghost-spring-boot-starter
```

Plugin id: com.ghostserializer.ghost version aligned with libraries.

Factual verification (aligned with code 1.2.0)

This manual was cross-checked against the ghost-serializer repository on the local working branch:

Manual claim	Source in repo
:---	:---
Registry sharding at 500 entries	GhostEmitterConstants.REGISTRY_CHUNK_SIZE = 500
Method fragmentation >40 fields	DEFAULT_CHUNK_SIZE = 40 in GhostEmitterConstants
Multi-branch up to 4 defaults	MAX_DEFAULT_BRANCH_COUNT = 4 in StandardEmitter.kt
maxCollectionSize Android 50k / JVM 1M / Nat 500k	GhostHeuristics.android.kt, .jvm.kt, .native.kt
maxWarmWriteBuffer 4 MB mobile / 8 MB JVM	same GhostHeuristics.*.kt files
maxDepth 255	GhostJsonConstants.MAX_DEPTH
416 tests in ciTestJvm (verified)	Gradle build/test-results XML, May 2026
642 tests ./gradlew ciTest on Linux	416 + 226 Android testDebugUnitTest
~874 with iOS on macOS	README: 642 + iosSimulatorArm64Test (~232)
Spring Boot 3.4.5 in tests	gradle/libs.versions.toml spring-boot = "3.4.5"
Ktor 2.3.11	gradle/libs.versions.toml ktor = "2.3.11"
iOS without ServiceLoader	Ghost.ios.kt → discoverRegistries() = emptyList()
JVM registry fast-path	Ghost.jvm.kt → Class.forName + ServiceLoader
List/Map at runtime	Ghost.kt → ListSerializer, MapSerializer
Set not at runtime	no SetSerializer in ghost-serialization (README may mention Set; verify release notes)

If you upgrade the Ghost version, cross-check these files again before trusting exact numbers.

Appendix: API Reference

This section documents the public API of Ghost Serialization, derived from the

KDoc comments in the source code (version 1.2.0).

A.1 Ghost — Main entry point

object Ghost in package com.ghost.serialization.

Central entry point for Ghost Serialization. Manages modular registry discovery

and serialization/deserialization across all platforms.

Serialization

Method	Description
:---	:---
serialize(sink, value)	Encodes value and writes the resulting JSON payload into the given Okio BufferedSink. Uses GhostJsonFlatWriter internally for a single bulk-write with no Okio segment overhead.
serialize(value)	Convenience alias for encodeToString.
encodeToString(value)	Serializes value to an in-memory JSON String. Writes to a flat contiguous byte buffer and performs a zero-copy string decode at the end.
encodeToBytes(value)	Serializes value to a UTF-8 JSON ByteArray. Skips intermediate string encoding/decoding.
encodeAndDiscard(value)	Serializes value discarding the output. Useful for JIT/ART priming without needing the resulting bytes.
encodeToSink(sink, value)	Alias for serialize(sink, value).
encodeToSink(sink, value, clazz)	Non-inline variant of encodeToSink for contexts where the type is only known as a KClass at runtime (e.g. Spring, Retrofit).

Deserialization

Method	Description
:---	:---
deserialize<T>(json)	Deserializes a JSON String into an instance of type T.
deserialize<T>(source)	Deserializes from an Okio BufferedSource stream. Reads all bytes eagerly into a reusable scratch buffer.
deserialize<T>(bytes)	Deserializes a UTF-8 JSON ByteArray into an instance of type T.

deserialize<T>(json, options)	**Advanced: Deserializes a JSON String with custom parser settings.**
deserialize<T>(source, options)	**Advanced: Deserializes from a BufferedSource with custom parser settings.**
deserialize<T>(bytes, options)	**Advanced: Deserializes a ByteArray with custom parser settings.**
decodeFromBytes(bytes, clazz, limit)	**Non-inline variant for reflection/framework contexts (Spring, Retrofit).**
decodeFromSource(source, clazz)	**Non-inline variant of deserialize for BufferedSource.**

Registry management

Method	Description
:---	:---
addRegistry(registry)	**Registers a GhostRegistry manually. Critical on iOS, Wasm, and JS targets where ServiceLoader is unavailable.**
getSerializer(clazz)	**Resolves the GhostSerializer for a given class. Checks primitives first, then the fast-path cache, then registered modules.**
getSerializer(type)	**Resolves the GhostSerializer for a KType, supporting generic types (List<T>, Map<K,V>).**
prewarm()	**Triggers eager loading and JIT/ART warm-up cycles for all registered serializers. Call at app startup for zero-latency first-run deserialization.**
throwError(message)	**Throws IllegalArgumentException. Utility for generated serializers.**

A.2 GhostRegistry — Module contract

interface GhostRegistry in package com.ghost.serialization.contract.

Registry interface for discovering and managing compiler-generated and custom serializers.

Implementations are typically generated automatically as module-level registries by the Ghost

compiler plugin, allowing reflection-free serializer lookup across all targets in a Kotlin

Multiplatform project.

Method	Description
:---	:---
getSerializer(clazz)	**Resolves a GhostSerializer for the given class. Returns null if not registered in this module.**
getAllSerializers()	**Returns all serializers registered in this module. Used by Ghost for eager loading and prewarm.**
prewarm()	**Eagerly initializes registry entries. No-op by default; can be overridden.**

registeredCount()	Returns the total number of serializers in this registry.

A.3 GhostJsonException — Parsing exception

class GhostJsonException in package com.ghost.serialization.exception.

Exception type thrown for JSON parsing/encoding errors.

To keep the failure path cheap (the parser may raise this exception in tight loops

while probing payloads), line and column are computed lazily — the O(N)

scan over the source bytes is only paid if the caller actually reads either property

or accesses message.

Property / Constructor	Description
:---	:---
path: String	The dot-separated JSON path where the error occurred. Defaults to "$" (root).
line: Int	The 1-indexed line number in the JSON source where the error occurred. Computed lazily.
column: Int	The 1-indexed column number in the JSON source where the error occurred. Computed lazily.
message	Full message: "<msg> [at line X, col Y, path Z]".
constructor(message, line, column, path)	Secondary constructor with an explicit error location.

A.4 JsonReaderOptions — Optimized field dispatch

class JsonReaderOptions in package com.ghost.serialization.parser.

Dispatch options for optimized JSON field identification. Uses a 4-byte hashing

engine to minimize collisions during field lookup.

rawBytes stores field names as raw ByteArray instead of Okio's ByteString,

so that verifyKeyMatch can compare bytes directly without virtual dispatch

or redundant bounds checks inside Okio's rangeEquals.

Factory	Description
:---	:---
JsonReaderOptions.of(vararg names)	Creates an optimized configuration with default hashing parameters (shift=0, multiplier=31).
JsonReaderOptions.of(shift, multiplier, vararg names)	Creates a configuration with custom hashing shift and multiplier values to minimize collisions for a specific field set.

A.5 @InternalGhostApi — Internal use annotation

annotation class InternalGhostApi in package com.ghost.serialization.

Marks declarations that are internal to Ghost Serialization.

Declarations annotated with this annotation are not intended for public use and may

change or be removed in future versions without notice. Code generated by the Ghost KSP

compiler plugin uses this annotation to access internal optimized helper functions.

Opt-in level: RequiresOptIn.Level.WARNING. User code that directly uses this API will receive a compiler warning.